Project Report: Snake Game

Submitted by: Rajat Prafful Sahuji

Student Roll No: 47

Student Enrollment ID: 2403111009

Submitted to: Dr. R. S. Khamitkar

Department: Electronics and Computer Engineering **Date:** April 8, 2026

---

1. Project Overview

This project is a Python-based Snake Game developed using the turtle graphics module. The player controls a snake on a 600×600 grid, navigating it to consume food items that appear at random positions. Each time the snake eats food, it grows longer, the score increases by 10 points, and the game speed gradually increases. The game tracks both the current score and the high score in real time. The game ends when the snake collides with the boundary walls or with its own body.

2. Problem Statement

Classic arcade games like Snake are widely used in academic settings to demonstrate fundamental programming concepts such as event-driven programming, collision detection, dynamic data structures, and real-time rendering. This project aims to implement a fully functional Snake Game in Python using only the built-in turtle module without any external game libraries to demonstrate how core Python constructs can be applied to build an interactive, visually appealing game with smooth gameplay mechanics.

3. Technology Stack

- **Programming Language:** Python 3.x
- **Graphics Module:** turtle (built-in)
- **Other Modules:** random, time
- **IDE:** Python IDLE / VS Code

4. Implementation

The implementation is structured around the following key components:

- **Screen Setup:** A 600×600 pink window is created using turtle.Screen(). Tracer is disabled for manual rendering control.
- **Snake Head:** A circular green turtle object acts as the snake's head. It starts at the center and responds to WASD keyboard inputs.

- **Food:** A red square turtle is placed at a random position. When the snake's head comes within 20 pixels of the food, it is eaten.
- **Segments:** Grey square turtles are appended to a list each time food is eaten. They follow the snake's path using a reverse-iteration update.
- **Score Display:** A hidden pen turtle writes the current score and high score at the top of the screen, updating after each food collection.
- **Collision Detection:** Boundary collision resets the snake and score. Self-collision (segment distance < 20) also triggers a reset.
- **Speed Scaling:** The delay variable starts at 0.1 seconds and decreases by 0.001 with every food item consumed, gradually increasing speed.

5. Code Snippet

Python

```
# Snake Game Python Turtle

import turtle, time, random


delay = 0.1

score = 0

high_score = 0


wn = turtle.Screen()

wn.title("Snake Game by Tanish Swapnil Shah")

wn.bgcolor("pink")

wn.setup(width=600, height=600)

wn.tracer(0)


head = turtle.Turtle()

head.shape("circle")

head.color("green")

head.penup()

head.goto(0,0)

head.direction = "stop"
```

```
food = turtle.Turtle()
food.shape("square")
food.color("red")
food.penup()
food.goto(0, 100)

segments = []
# Movement functions (go_up, go_down, go_left, go_right)
# Keyboard bindings: W/A/S/D

while True: # Main game loop
    wn.update()
    # Border collision check
    # Food collision -> grow
    # Segment position update
    # Self-collision check
    time.sleep(delay)
```

6. Sample Test Cases

Action / Event	**Expected Output**
Snake eats food	Score increases by 10; new grey segment added; food repositions randomly
Snake hits boundary wall	Snake resets to center; score resets to 0; segments cleared
Snake collides with itself	Snake resets to center; score resets to 0; segments cleared
High score is beaten	High score updates to the new score value on screen
Multiple food items eaten	Snake grows longer, delay decreases, speed increases progressively

7. Challenges Faced

- **Segment Movement Logic:** Correctly updating segment positions required iterating in reverse order so each segment follows the one ahead of it without overwriting position data prematurely.
- **Collision Sensitivity:** Turtle's distance-based collision (threshold < 20 pixels) required careful tuning to avoid false positives while keeping gameplay responsive.
- **Speed Management:** Balancing the delay decrement (0.001 per food) ensures the game stays playable for longer sessions without becoming uncontrollably fast too quickly.
- **Screen Rendering:** Disabling the tracer and manually calling wn.update() each frame was necessary to prevent flickering and ensure smooth animation.

8. Results and Observations

The Snake Game runs successfully with smooth real-time movement and accurate collision detection. The scoring system correctly increments by 10 for each food item consumed and persistently tracks the high score across multiple rounds within the same session. The snake body grows as expected and the progressive speed increase provides a natural difficulty curve. Both boundary and self-collision resets work reliably, returning the game to its initial state without requiring a restart. The program demonstrates effective use of Python's turtle, time, and random modules in an event-driven game loop.

Project Submission

Repository Link: https://github.com/rajatsahuji6-ship-it/snake-game

Student Signature: _____ **Date:** April 8, 2026 **Guide Signature:** _____